

14.5 类设计例

14.5.1 多边形

设计并定义一个表示平面上多边形的类，该类的主要功能为计算多边形的周长和面积，读取顶点数量以及存取各顶点坐标。

该类的数据成员包括多边形的边数以及各个顶点的坐标；主要功能包括计算多边形的周长和面积，以及读取顶点数和各顶点坐标。

计算周长和面积可通过实例方法 `perimeter()` 和 `area()` 实现；读取顶点数量和存取各顶点坐标可通过 `getter/setter` 函数实现；为了存储各顶点坐标数据，可以选择使用堆区的动态数组，也可以采用标准库中的 `vector` 容器。由于数据成员都是普通内置类型和对象，因此无需显式定义析构函数、拷贝构造函数和赋值操作符，使用系统缺省(默认)提供的即可满足定义需求。此外，为了便于计算周长与面积，引入两个辅助函数：计算平面上两点距离和计算三角形面积，考虑到这两个函数的计算方法是通用的，因此将这两个函数定义为类方法。具体代码如下：

```

1  //Polygon.h
2  #pragma once
3  #include <initializer_list>
4  #include <vector>
5
6  struct Point {
7      double x;
8      double y;
9  };
10
11 class Polygon {
12 public:
13     Polygon( Point pts[ ], size_t size );
14     Polygon( std::initializer_list<Point> list );
15 public:
16     double perimeter( ) const;
17     double area( ) const;
18 public: //getter
19     size_t sides( ) const    { return mSize; }
20     const std::vector<Point>& vertexes( ) const    { return mVertexes; }
21     const Point& operator[]( size_t index ) const { return mVertexes[ index ]; }
22     Point& operator[]( size_t index )            { return mVertexes[ index ]; }
23 private:
24     static double distance( const Point& pt1, const Point& pt2 );

```

```

25     static double triangleArea(const Point& pt1,const Point& pt2,const Point& pt3 );
26 private:
27     size_t          mSize;          //顶点数
28     std::vector<Point> mVertexes;    //各顶点
29 };

```

```

1  //Polygon.cpp
2  #include <initializer_list>
3  #include <cassert>
4  using namespace std;
5  #include "Polygon.h"
6  //构造函数
7  Polygon::Polygon( Point pts[ ], size_t size )
8      :mSize( size ), mVertexes( mSize ) {
9      assert( mSize >= 3 );
10     mVertexes.assign( pts, pts + mSize );
11 }
12 //构造函数
13 Polygon::Polygon( std::initializer_list<Point> list )
14     :mSize( list.size( ) ), mVertexes( mSize ) {
15     assert( mSize >= 3 );
16     mVertexes.assign( list.begin( ), list.end( ) );
17 }
18 //计算周长
19 double Polygon::perimeter( ) const    {
20     double result = 0.0;
21     for (int i = 0; i < mSize - 1; ++i) {
22         result += distance( mVertexes[ i ], mVertexes[ (size_t)i + 1 ] );
23     }
24     result += distance( mVertexes.front( ), mVertexes.back( ) );
25     return result;
26 }
27 //计算面积
28 double Polygon::area( ) const        {
29     double result = 0.0;
30     const Point& origin = mVertexes.front( );
31     for (int i = 1; i < mSize - 1; ++i) {
32         result += triangleArea( origin, mVertexes[i], mVertexes[(size_t)i + 1] );
33     }
34     return result;
35 }
36 //类方法：两点之间距离
37 double Polygon::distance( const Point& p1, const Point& p2 )    {

```

```

38     return sqrt( ( p1.x - p2.x ) * ( p1.x - p2.x )
39                 + ( p1.y - p2.y ) * ( p1.y - p2.y ) );
40 }
41 //类方法: 海伦公式计算三角形面积
42 double Polygon::triangleArea( const Point& p1, const Point& p2, const Point& p3 ) {
43     double d1 = distance( p1, p2 );
44     double d2 = distance( p2, p3 );
45     double d3 = distance( p3, p1 );
46     double s = ( d1 + d2 + d3 ) / 2;
47     return sqrt( s * ( s - d1 ) * ( s - d2 ) * ( s - d3 ) );
48 }

```

```

1  //主函数
2  #include <iostream>
3  #include "Polygon.h"
4  int main( ) {
5      Point pts[ 4 ] { {0,0},{0,1},{1,0},{1,-1} };
6      Polygon p1 { pts, 4 };
7      std::cout << "p1的边数 = " << p1.sides( ) << std::endl;
8      std::cout << "p1的周长 = " << p1.perimeter( ) << std::endl;
9      std::cout << "p1的面积 = " << p1.area( ) << std::endl;
10     std::cout << "p1的各项点坐标: \n";
11     for ( auto& item : p1.vertexes( ) )
12         std::cout << "(x,y) = (" << item.x << "," << item.y << ")\n";
13
14     Polygon p2 { {0,0},{0,1},{1,0},{ 1,-2 } };
15     std::cout << "p2的边数 = " << p2.sides( ) << std::endl;
16     std::cout << "p2的周长 = " << p2.perimeter( ) << std::endl;
17     std::cout << "p2的面积 = " << p2.area( ) << std::endl;
18 }

```

输出为:

```

p1 的边数 = 4
p1 的周长 = 4.82843
p1 的面积 = 1
p1 的各项点坐标:
(x,y) = (0,0)
(x,y) = (0,1)
(x,y) = (1,0)
(x,y) = (1,-1)
p2 的边数 = 4
p2 的周长 = 6.65028
p2 的面积 = 1.5

```

14.5.2 伪随机数

伪随机数生成在计算机科学领域应用广泛,伪随机数是通过数学方法生成的,虽然看起来是随机的,但实际上是可预测的。线性同余算法(Linear Congruential Generator, LCG)是最早应用的伪随机数生成算法,其原理公式为: $X_{n+1} = (M * X_n + N) \% K$ 。

其中, X 是伪随机数序列, M 是乘数, N 是增量, K 是模数, X_0 称为随机序列种子。给定 M 、 N 、 K 和 X_0 , 就能够得到无限长度的随机数序列 $X_1, X_2, \dots, X_n, \dots$, 这个序列中的每个 X_n 也就是伪随机数了。下面就使用线性同余算法定义一个伪随机数生成器类并使用该类:

```

1  //LCGRandom.h
2  #pragma once
3  #include <iostream>
4  class LCGRandom {
5  public:
6      LCGRandom( unsigned long seed ) :mSeed( seed ) { }
7      LCGRandom( const LCGRandom& ) = delete;
8      LCGRandom& operator=( const LCGRandom& ) = delete;
9  public:
10     //[low,high)之间的 int 型整数
11     int randInt( int min = 0, int max = INT_MAX )
12         { return next( ) % ( max - min ) + min; }
13
14     //[low,high)之间的 long 型整数
15     long randLong( long min = 0, int max = LONG_MAX )
16         { return next( ) % ( max - min ) + min; }
17
18     //(0,1)之间的随机双精度浮点
19     double randReal( ) { return ( (double)next( ) ) / ULONG_MAX; };
20 private:
21     void change( ) { mSeed = ( M * mSeed + N ) % ULONG_MAX; }
22     long next( ) {
23         int loops = mSeed % 3;
24         for (int i = 0; i <= loops; i++)
25             change( ); //让随机数分布更均匀
26         return mSeed / 2;
27     }
28 private:
29     static const unsigned long M = 99985057; //通常选大质数
30     static const unsigned long N = 10002803; //通常选大质数
31     unsigned long mSeed;

```

```
32  };
```

```
1  //TestLCG.cpp
2  #include <iostream>
3  using namespace std;
4  #include "LCGRandom.h"
5  int main( ) {
6      LCGRandom rand( 97 );
7      //输出 10 个整数
8      for (int i = 0; i < 10; i++)
9          cout << rand.randInt( ) << "\n";
10     cout << "\n";
11     //输出 10 个[-128, 127]之间的整数
12     for (int i = 0; i < 10; i++)
13         cout << rand.randInt( -128, 128 ) << "\n";
14     cout << "\n";
15     //输出 10 个(0,1)之间的双精度浮点数
16     for (int i = 0; i < 10; i++)
17         cout << rand.randReal( ) << "\n";
18 }
```

伪随机数发生器可以应用在多种场景。下面就是一个使用伪随机数进行加密解密的示例：

```
1  //Cryptor.h
2  #include "LCGRandom.h"
3  class Cryptor {
4  public:
5      static void encryptor( unsigned long key, char* str ) {
6          LCGRandom rand( key );
7          size_t size = strlen( str );
8          for (int i = 0; i < size; i++)
9              str[ i ] ^= rand.randInt( 0, 255 );    //异或操作
10     }
11     static void decryptor( unsigned long key, char* str ) {
12         LCGRandom rand( key );
13         size_t size = strlen( str );
14         for (int i = 0; i < size; i++)
15             str[ i ] ^= rand.randInt( 0, 255 );    //异或操作
16     }
17 };
```

```
1  //TestCryptor.cpp
2  #include <iostream>
```

```

3  #include <iomanip>
4  using namespace std;
5  int main( ) {
6      char text[ ] = "面向对象程序设计";
7      size_t size = strlen( text );
8
9      cout << "明文字符串\t: " << text;
10     cout << "\n 明文的各字节\t: ";
11     for (int i = 0; i < size; i++)
12         cout << setfill('0') << setw(2) << hex << (int) (text[ i ] & 0xFF) << " ";
13
14     Cryptor::encryptor( 1234, text );
15     cout << "\n 密文的各字节\t: ";
16     for (int i = 0; i < size; i++)
17         cout << setfill('0') << setw(2) << hex << (int) (text[ i ] & 0xFF) << " ";
18
19     Cryptor::decryptor( 1234, text );
20     std::cout << "\n 解密后的各字节\t: ";
21     for (int i = 0; i < size; i++)
22         cout << setfill('0') << setw(2) << hex << (int) (text[ i ] & 0xFF) << " ";
23     std::cout << "\n 解密后的字符串\t: " << text << "\n";
24 }

```

输出为:

明文字符串	: 面向对象程序设计
明文的各字节	: c3 e6 cf f2 b6 d4 cf f3 b3 cc d0 f2 c9 e8 bc c6
密文的各字节	: df bb f9 48 12 93 23 77 6f 45 cc e4 e8 98 9c 2a
解密后的各字节	: c3 e6 cf f2 b6 d4 cf f3 b3 cc d0 f2 c9 e8 bc c6
解密后的字符串	: 面向对象程序设计

`Cryptor` 类的设计方式不是唯一的, 示例中的 `Cryptor` 类通过类方法封装了加密和解密功能, 也可以使用实例方法定义加密和解密功能。

14.5.3 分页器

分页器是应用程序中常见的功能组件, 主要用于将大量检索结果划分为多个逻辑页面, 从而提升用户浏览和操作数据的效率与体验。其典型工作流程如下: 服务端首先根据查询条件获取原始数据集, 并结合分页器进行数据切片; 随后, 将当前的页面数据、分页器, 返回给客户端; 客户端据此渲染页面数据及分页器。当用户交互操作分页器时, 会向服务端发起请求, 服务端再次利用分页信息执行分页查询并返回对应页面的数据及更新后的分页器, 以此实现分页展示。

例如一个总页码 100 页且当前为第 4 页的分页器可能展示为如下形式：

< 1 2 3 (4) 5 6 7 ... 100 > 每页 20 条 转到[]页

一个分页器的主要功能包括：导航按钮，通过上一页、下一页、上 N 页、下 N 页、首页、末页等按钮，实现页面之间的切换；页码列表，显式当前页码和总页码，用户可通过点击页码实现页面跳转；页面条目数选择，用户可以指定每个页面的最大显示条数；快速跳转，允许直接跳转到用户指定目标页码。具体的分页器类可如下定义：

```

1  class Paginate {
2      static const  size_t N = 5;
3  public:
4      void first( )    { mCurPage = 1; }
5      void last( )     { mCurPage = mCountPages; }
6      void prev( )      { if (mCurPage > 1) --mCurPage; }
7      void next( )      { if (mCurPage < mCountPages) ++mCurPage; };
8      void prevN( )     { mCurPage = ( mCurPage <= N ? 1 : mCurPage - N ); }
9      void nextN( )
10         { mCurPage = ( mCurPage + N >= mCountPages ? mCountPages : mCurPage + N ); }
11     void go( int page )
12         { if (page >= 1 && page <= mCountPages) mCurPage = page; }
13 public:
14     int     countItems( ) const          { return mCountItems;    }
15     void    setCountItems( int total )    { mCountItems = total;    }
16     int     pageItems( ) const           { return mPageItems;     }
17     void    setPageItems( int items )     { mPageItems = items;     }
18     int     countPages( ) const           { return mCountPages;    }
19     void    setCountPages( int pages )    { mCountPages = pages;    }
20     int     curPage( ) const              { return mCurPage;      }
21     void    setCurPage( int page )       { mCurPage = page;      }
22 private:
23     int mCountItems = 1;    //总数据条数
24     int mPageItems = 20;    //每页条数
25     int mCountPages = 1;    //总页码
26     int mCurPage = 1;      //当前页码
27 };

```

这里 Paginate 类的数据成员定义为私有的，而通过公有的 getter/setter 函数存取这些它们。下面再给出模拟使用 Paginate 类的程序和结果，其中 Server 类模拟服务端，Client 模拟客户端：

```

1  #include <iostream>
2  #include <iomanip>
3  #include <list>

```

```

4  using namespace std;
5  class Server {
6  public:
7      //模拟查询检索
8      std::list<int> query( Paginate& pgr ) {
9          int total = 190; //模拟总条数 190
10         pgr.setCountItems( total );
11
12         int pages = total > 0 ? ( total - 1 ) / pgr.pageItems( ) + 1 : 1;
13         pgr.setCountPages( pages ); //至少 1 页
14
15         //模拟查询出 pgr.curPage()页的数据
16         return std::list<int>{1, 2, 3, 6};
17     }
18 };
19
20 class Client {
21 public:
22     void update( const std::list<int>& list, const Paginate& pgr ) const {
23         showList( list );
24         showPaginate( pgr );
25     }
26 private:
27     void showList( const std::list<int>& list ) const { /* 略 */ }
28     void showPaginate( const Paginate& pgr ) const {
29         int from = max( 1, pgr.curPage( ) - 3 );
30         int to = min( pgr.curPage( ) + 3, pgr.countPages( ) );
31
32         cout << " < ";
33         for (int i = from; i <= to; ++i) {
34             if (i == pgr.curPage( )) cout << " (" << i << ")";
35             else cout << setw( 4 ) << i;
36         }
37         if ( to < pgr.countPages( ) - 1 ) cout << setw( 4 ) << "...";
38         if ( to < pgr.countPages( )) cout << setw( 4 ) << pgr.countPages( );
39
40         cout << " > ";
41         cout << " 每页 " << setw( 4 ) << pgr.pageItems( ) << "条";
42         cout << " \t 转到[ " << setw( 4 ) << "" << "]"页\n";
43     }
44 };
45 int main( ) {
46     Server srv;
47     Client cli;

```



```

48     Paginate pgr;
49     //显示第 1 页
50     auto list1 = srv.query( pgr );
51     cli.update( list1, pgr );
52     //显示第 2 页
53     pgr.next( );
54     auto list2 = srv.query( pgr );
55     cli.update( list2, pgr );
56     //显示第 2+5=7 页
57     pgr.nextN( );
58     auto list3 = srv.query( pgr );
59     cli.update( list3, pgr );
60 }

```

运行结果为:

```

< (1)  2  3  4 ... 10 >  每页  20 条  转到[    ]页
<   1 (2)  3  4   5 ... 10 >  每页  20 条      转到[    ]页
<   4  5  6 (7)  8  9 10 >  每页  20 条      转到[    ]页

```

14.5.4 骰子

现定义一个骰子类，并模拟掷骰子的行为结果。一个骰子具有 6 个面，各面的数值分别为 1、2、3、4、5、6，且任意一对相对面的数值之和恒为 7。在掷骰子的过程中，各面的数值保持不变，面与面之间的相对位置关系也是固定的。对于常见的骰子而言，数值 1、2、3 所在的三个相邻面，其相对位置通常符合右手坐标系规则。

每次掷骰子的结果表现为朝上一面的数值；整个投掷过程可以理解为骰子围绕 X 轴、Y 轴、Z 轴进行一系列随机旋转的结果。假设所使用的三维坐标系（XYZ 轴）也为右手系。骰子的六个面分别定义为：上面(Top)、底面(Bottom)、左面(Left)、右面(Right)、前面(远离观察者的一侧，Front)和后面(靠近观察者的一侧，Back)。随机数的生成既可以使用 `rand()` 函数，也可以借助标准库中的 `Random` 模块实现。

```

1  // Dice.h
2  #pragma once
3  #include <unordered_map>
4  #include <random>
5
6  enum class Direction { TOP = 0, RIGHT, FRONT, BOTTOM, LEFT, BACK };
7  class Dice {

```

```

8  public:
9      int value( Direction dir = Direction::TOP ) const { return mValues.at( dir ); }
10     int cast( ); //掷骰子
11 #ifdef _DEBUG
12 public:
13 #else
14 private:
15 #endif
16     void rotateX( ); //沿 X 轴的正方向, 顺时针旋转 90 度
17     void rotateY( ); //沿 Y 轴的正方向, 顺时针旋转 90 度
18     void rotateZ( ); //沿 Z 轴的正方向, 顺时针旋转 90 度
19 private:
20     static std::mt19937_64 & randGen( ); //随机数发生器
21 private:
22     //各面的初始值。也可不用 unordered_map, 而用数组实现。
23     std::unordered_map<Direction, int> mValues {
24         { Direction::TOP, 1 },    { Direction::RIGHT, 2 },
25         { Direction::FRONT, 3 }, { Direction::BOTTOM, 6 },
26         { Direction::LEFT, 5 },   { Direction::BACK, 4 },
27     };
28 };

```

```

1  // Dice.cpp
2  #include "Dice.h"
3  //随机数发生器
4  std::mt19937_64 & Dice::randGen( ) {
5      static std::random_device rd;
6      static std::mt19937_64 generator( rd( ) );
7      return generator;
8  }
9  //模拟投掷过程
10 int Dice::cast( ) {
11     static std::uniform_int_distribution<long> dist( 0, 3 );//[0,3]均匀分布
12
13     int xTimes = dist( randGen( ) ); //绕 X 轴旋转 xTimes 次 90 度
14     int yTimes = dist( randGen( ) ); //绕 Y 轴旋转 yTimes 次 90 度
15     int zTimes = dist( randGen( ) ); //绕 Z 轴旋转 zTimes 次 90 度
16
17     for (int i = 0; i < xTimes; i++) rotateX( );
18     for (int i = 0; i < yTimes; i++) rotateY( );
19     for (int i = 0; i < zTimes; i++) rotateZ( );
20
21     return mValues.at( Direction::TOP ); //返回结果值

```

```

22 }
23 //沿 X 轴的正方向, 顺时针旋转 90 度, LEFT, RIGHT 的值不变
24 void Dice::rotateX( ) {
25     int temp = mValues.at( Direction::TOP );
26     mValues[ Direction::TOP ] = mValues.at( Direction::BACK );
27     mValues[ Direction::BACK ] = mValues.at( Direction::BOTTOM );
28     mValues[ Direction::BOTTOM ] = mValues.at( Direction::FRONT );
29     mValues[ Direction::FRONT ] = temp;
30 }
31 //沿 Y 轴的正方向, 顺时针旋转 90 度, FRONT, BACK 的值不变
32 void Dice::rotateY( ) {
33     int temp = mValues.at( Direction::TOP );
34     mValues[ Direction::TOP ] = mValues.at( Direction::RIGHT );
35     mValues[ Direction::RIGHT ] = mValues.at( Direction::BOTTOM );
36     mValues[ Direction::BOTTOM ] = mValues.at( Direction::LEFT );
37     mValues[ Direction::LEFT ] = temp;
38 }
39 //沿 Z 轴的正方向, 顺时针旋转 90 度, TOP, BOTTOM 的值不变
40 void Dice::rotateZ( ) {
41     int temp = mValues.at( Direction::LEFT );
42     mValues[ Direction::LEFT ] = mValues.at( Direction::BACK );
43     mValues[ Direction::BACK ] = mValues.at( Direction::RIGHT );
44     mValues[ Direction::RIGHT ] = mValues.at( Direction::FRONT );
45     mValues[ Direction::FRONT ] = temp;
46 }

```

```

1 //测试主程序
2 #include <iostream>
3 using namespace std;
4 #include "Dice.h"
5
6 #ifdef _DEBUG
7 void showDice( const Dice & dice ) {
8     cout << "TOP:\t" << dice.value( Direction::TOP ) << endl;
9     cout << "RIGHT:\t" << dice.value( Direction::RIGHT ) << endl;
10    cout << "FRONT:\t" << dice.value( Direction::FRONT ) << endl;
11    cout << "BACK:\t" << dice.value( Direction::BACK ) << endl;
12    cout << "LEFT:\t" << dice.value( Direction::LEFT ) << endl;
13    cout << "BOTTOM:\t" << dice.value( Direction::BOTTOM ) << endl;
14    cout << endl;
15 }
16 #endif
17

```

```

18 int main( ) {
19     Dice dice;
20     #ifdef _DEBUG
21         dice.rotateX( );
22         showDice( dice );
23         dice.rotateY( );
24         showDice( dice );
25         dice.rotateZ( );
26         showDice( dice );
27     #else
28         cout << "result of cast() = " << dice.cast( ) << endl;
29         cout << "result of cast() = " << dice.cast( ) << endl;
30         cout << "result of cast() = " << dice.cast( ) << endl;
31         cout << "result of cast() = " << dice.cast( ) << endl;
32     #endif
33 }

```

14.5.5 创建方法

一个类中可以有多个重载的构造函数，用户在实例化该类对象时，应选择适当的构造函数。但当构造函数较多，特别是参数表复杂时，选用正确的构造函数需要清晰地理解各参数的语义，而有时语义不是显而易见的，这时就会给该类的用户造成困扰。例如下面的气球类：

```

1 class Balloon {
2 public:
3     Balloon( ) { }           //构造普通气球
4     Balloon( int color ) { } //构造彩色气球
5     Balloon( Gas & gas ) { } //构造充气气球
6 };

```

Balloon 类有三个构造函数，它们的语义还算简单的，若更复杂一些，常常需要通过注释进行说明。最好的解决办法是让构造函数具有更易理解的名字，也就是通过函数名让函数语义自明，但构造函数又要求名字必须是类名。为此，可考虑使用创建方法重新设计，如：

```

1 class Balloon {
2 public:
3     static Balloon createNormalBalloon( )           { return Balloon( ); }
4     static Balloon createColorBalloon( int color ) { return Balloon(color); }
5     static Balloon createGasedBalloon( Gas & gas ) { return Balloon(gas); }
6 private:
7     Balloon( ) { }           //构造普通气球

```

```

8      Balloon( int color ) { } //构造彩色气球
9      Balloon( Gas & gas ) { } //构造充气气球
10 };

```

这里三个公有的创建方法，通过不同的函数名可以清晰地表示创建各种对象的语义，其功能就是最初的构造函数功能。

14.5.6 枚举对象

在 C++ 中，无论是普通枚举还是枚举类，其数据类型都是基于整型的。如：

```

1  enum Season { SPRING = 0, SUMMER, AUTUMN, WINTER }; //int 型
2  enum class Color : unsigned long { //unsigned long
3      RED = 0xFF0000,
4      GREEN = 0x00FF00,
5      BLUE = 0x0000FF,
6  };

```

尽管有的面向对象语言支持枚举对象，但 C++ 语言目前还不支持，不过我们可以通过自定义类来实现枚举对象。例如：

```

1  // Weekday.h
2  #include <string>
3  class Weekday {
4  private:
5      enum class ID { MON = 1, TUE, WED, THU, FRI, SAT, SUN = 0 };
6      Weekday( ID id ) : mID( id ) { }
7  public:
8      int id( ) const { return (int)mID; }
9      const std::string& name( ) const {
10          static const std::string names[ ]
11              { "SUN", "MON", "TUE", "WED", "THU", "FRI", "SAT" };
12          return names[ (int)mID ];
13      }
14      const std::string& toString( ) const {
15          static const std::string names[ ]
16              { "星期日", "星期一", "星期二", "星期三", "星期四", "星期五", "星期六" };
17          return names[ (int)mID ];
18      }
19  public: //类变量表示枚举对象
20      static const Weekday Mon;
21      static const Weekday Tue;
22      static const Weekday Wed;
23      static const Weekday Thu;

```

```

24     static const Weekday Fri;
25     static const Weekday Sat;
26     static const Weekday Sun;
27 private:
28     ID    mID;
29 };

```

```

1  //TestWeekday.cpp
2  const Weekday Weekday::Sun( Weekday::ID::SUN );
3  const Weekday Weekday::Mon( Weekday::ID::MON );
4  const Weekday Weekday::Tue( Weekday::ID::TUE );
5  const Weekday Weekday::Wed( Weekday::ID::WED );
6  const Weekday Weekday::Thu( Weekday::ID::THU );
7  const Weekday Weekday::Fri( Weekday::ID::FRI );
8  const Weekday Weekday::Sat( Weekday::ID::SAT );
9
10 int main( ) {
11     auto d0 = Weekday::Sun;
12     std::cout << d0.id( ) << std::endl;
13     std::cout << d0.name( ) << std::endl;
14     std::cout << d0.toString( ) << std::endl;
15
16     auto d1 = Weekday::Mon;
17     std::cout << d1.id( ) << std::endl;
18     std::cout << d1.name( ) << std::endl;
19     std::cout << d1.toString( ) << std::endl;
20 }

```

上边的示例中，使用类变量表示枚举对象，如果枚举的对象更多，这种方式就不够方便了。此时，也可以使用容器来存储各枚举对象。例如：

```

1  #include <unordered_map>
2  class Weekday {
3  public:
4      enum class ID { MON = 1, TUE, WED, THU, FRI, SAT, SUN = 0 };
5      static const Weekday& day( ID id ) {
6          static const std::unordered_map<Weekday::ID, Weekday> sDays {
7              {Weekday::ID::SUN, Weekday( Weekday::ID::SUN )},
8              {Weekday::ID::MON, Weekday( Weekday::ID::MON )},
9              {Weekday::ID::TUE, Weekday( Weekday::ID::TUE )},
10             {Weekday::ID::WED, Weekday( Weekday::ID::WED )},
11             {Weekday::ID::THU, Weekday( Weekday::ID::THU )},
12             {Weekday::ID::FRI, Weekday( Weekday::ID::FRI )},

```

```

13         {Weekday::ID::SAT, Weekday( Weekday::ID::SAT )}
14     };
15     return sDays.at( id );
16 }
17 public:
18     int id( ) const { return (int)mID; }
19     const std::string& name( ) const {
20         static const std::string names[ ]
21             { "SUN", "MON", "TUE", "WED", "THU", "FRI", "SAT" };
22         return names[ (int)mID ];
23     }
24     const std::string& toString( ) const {
25         static const std::string names[ ]
26             { "星期日", "星期一", "星期二", "星期三", "星期四", "星期五", "星期六" };
27         return names[ (int)mID ];
28     }
29 private:
30     Weekday( ID id ) :mID( id ) { }
31 private:
32     ID    mID;
33 };

```

此外，还可以使用创建方法创建各枚举对象。例如下面的性别类：

```

1  //Gender.h
2  #pragma once
3  #include <string>
4  enum class Lang { CHN = 0, ENG };
5
6  class Gender {
7  public:
8      enum class Type { MALE = 0, FEMALE = 1 };
9  public:
10     static const Gender & createMale( ) { //创建方法
11         static const Gender male { Gender::Type::MALE };
12         return male;
13     }
14     static const Gender & createFemale( ) { //创建方法
15         static const Gender female { Gender::Type::FEMALE };
16         return female;
17     }
18 private:
19     Gender( Type t ) :mType( t ) { }
20     Gender( const Gender& ) = delete;           //禁止拷贝

```

```

21     Gender( Gender&& ) = delete;           //禁止移动拷贝
22     Gender& operator=( const Gender& ) = delete; //禁止赋值
23 public:
24     const std::string & toString( Lang lang ) const {
25         static const std::string chnNames[ 2 ] { "男", "女" };
26         static const std::string engNames[ 2 ] { "Male", "Female" };
27         if (lang == Lang::ENG) return chnNames[ (size_t)mType ];
28         else return engNames[ (size_t)mType ];
29     }
30 private:
31     Type mType;
32 };

```

```

1  //测试主程序
2  #include <iostream>
3  #include "Gender.h"
4  int main( ) {
5      auto & g1 = Gender::createMale( );
6      std::cout << "g1 = " << g1.toString( Lang::CHN ) << std::endl; //男
7      std::cout << "g1 = " << g1.toString( Lang::ENG ) << std::endl; //Male
8
9      auto & g2 = Gender::createFemale( );
10     std::cout << "g2 = " << g2.toString( Lang::CHN ) << std::endl; //女
11     std::cout << "g2 = " << g2.toString( Lang::ENG ) << std::endl; //Female
12 }

```

输出结果为:

```

g1 = 男
g1 = Male
g2 = 女
g2 = Female

```

14.5.7 单例

一个类通常可以创建多个实例,但在某些场景下,我们希望整个程序运行期间该类仅存在一个实例,这个实例称为单例。典型的例子如各种管理器类,用于统一管理资源、配置或服务,避免了重复创建和状态不一致的问题。

以下定义的 Singleton 类通过代码层面的控制,确保用户最多只能创建一个实例:

```

1  #include <iostream>

```



```

2  class Singleton {
3  public:
4      static Singleton& getInstance( ) {
5          static Singleton instance;
6          return instance;
7      }
8      ~Singleton( ) = default;
9  private:
10     Singleton( ) = default;
11     Singleton( const Singleton& ) = delete;          // 禁止拷贝
12     Singleton( Singleton&& ) = delete;              // 禁止移动拷贝
13     Singleton& operator=( const Singleton& ) = delete; // 禁止赋值
14 public:
15     int test( )          { return 100; }
16     int test( ) const    { return 200; }
17     /* 其它成员函数略 */
18 private:
19     /* 其它数据成员数略 */
20 };
21
22 int main( ) {
23     Singleton& obj1 = Singleton::getInstance( );
24     std::cout << obj1.test( ) << std::endl; // 输出 100
25
26     const Singleton& obj2 = Singleton::getInstance( );
27     std::cout << obj2.test( ) << std::endl; // 输出 200
28 }

```

从代码可以看出,通过将构造函数设为私有,禁止了类外部直接创建实例;同时禁用拷贝构造、移动拷贝和赋值操作,防止实例被复制或赋值,从而避免产生多个实例。用户只能通过公有的静态成员函数 `getInstance()` 来获取唯一的全局实例。

当前实现中,使用静态局部变量来存储单例对象,若希望将单例对象分配在堆上(即动态内存),可改为定义一个静态局部指针变量,并在首次调用 `getInstance()` 时动态创建实例,在程序结束前通过静态方法 `releaseInstance()` 释放堆区对象,具体实现可自行完成。考虑堆区指针释放的安全性问题,也可通过定义静态的 `shared_ptr<Singleton>` 或 `unique_ptr<Singleton>` 类对象来解决。例如:

```

1  #include <memory>
2  class Singleton {
3  public:
4      static Singleton& getInstance( ) {
5          static std::unique_ptr<Singleton> instance { new Singleton( ) }; //唯一指针

```

```

6         return *instance;
7     }
8     ~Singleton( ) = default;
9     private:
10        Singleton( ) = default;
11        Singleton( const Singleton& ) = delete;           // 禁止拷贝
12        Singleton( Singleton&& ) = delete;               // 禁止移动拷贝
13        Singleton& operator=( const Singleton& ) = delete; // 禁止赋值
14    public:
15        int test( )          { return 100; }
16        int test( ) const    { return 200; }
17        /* 其它成员函数略 */
18    private:
19        /* 其它数据成员数略 */
20 };

```

定义了单例类后, 无需再以参数形式传递该类对象, 也不建议将其作为其他类的数据成员。当需要使用单例对象时, 可直接通过 `getInstance()` 静态方法获取, 确保了访问的统一性和实例的唯一性。

14.5.8 注册课程

现模拟实现学生的选课系统。一个学生(Student)有姓名和选课表(Schedule), 一门课程有名称、课程 ID 和选课学生的花名册(Roster)。一个选课表(Schedule)拥有该学生全部的所选课程名称, 一个花名册(Roster)拥有选该课程的全部学生名单。另有一个 Register 类, 该类中的 `enroll` 行为实现学生的选课。

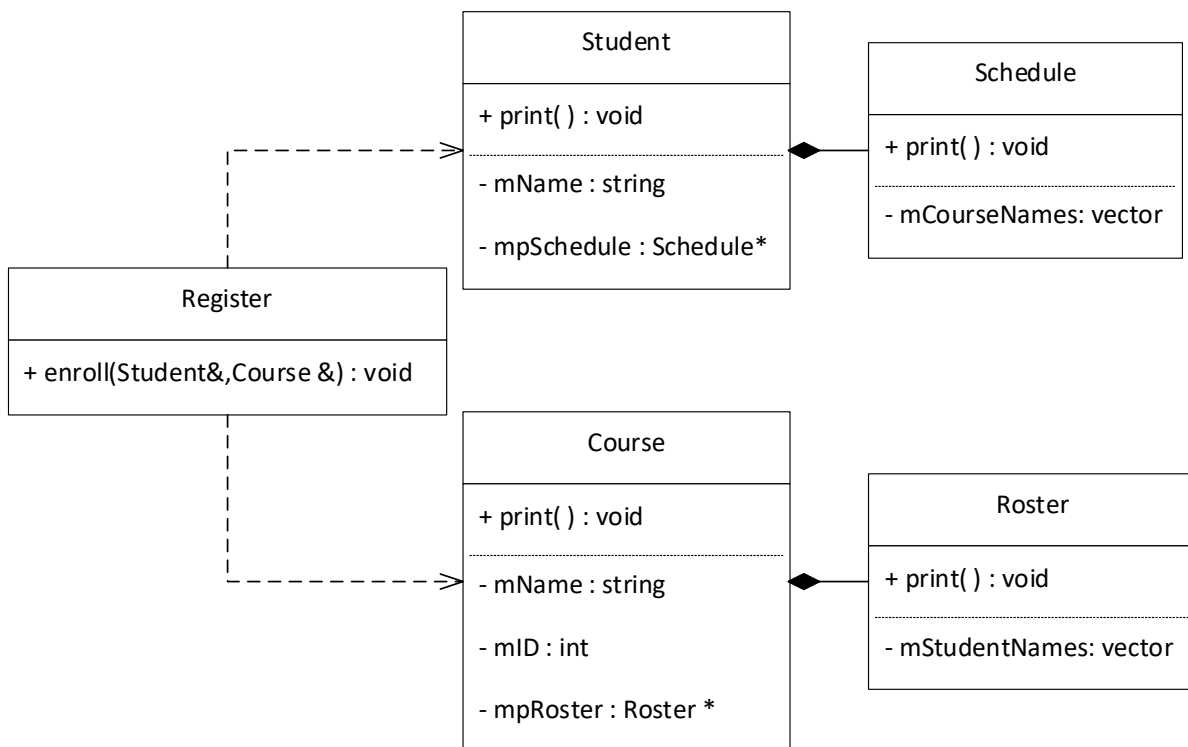


图 14-1 学生选课系统中各类之间的关系

Student 组合 Schedule, 所以 Student 负责创建和销毁 Schedule, 可以在 Student 类的构造和析构过程中实现此功能; 同理, 在 Course 类的构造和析构中实现花名册(Roster)的创建和销毁。

具体的代码如下:

```

1  //Student、Course、Schedule、Roster、Register 类的定义
2  #pragma once
3  #include <iostream>
4  #include <string>
5  #include <vector>
6  class Schedule; //学生选课表
7  //学生
8  class Student {
9  public:
10     Student( const std::string& name );
11     ~Student( );
12     void print( ) const;
13 public:
14     const std::string& name( ) const { return mName; }
15     Schedule* shedule( ) const { return mpSchedule; }
16 private:
17     std::string mName; //姓名
18     Schedule* mpSchedule; //选课表
19 };
  
```

```

20 //课程
21 class Course {
22 public:
23     Course( const std::string& courseName,int id );
24     ~Course( );
25     void print( ) const;
26 public:
27     const std::string& name( ) const { return mName; }
28     int id( ) const { return mID; }
29     Roster* roster( ) const { return mpRoster; }
30 private:
31     std::string      mName;          //课程名
32     const int        mID;            //课程 ID
33     Roster* mpRoster;                //学生名单
34 };
35 //学生的选课表
36 class Schedule {
37 public:
38     void addCourse( Course& course );
39     void print( ) const;
40 private:
41     std::vector<std::string> mCourseNames { }; //课程名称列表
42 };
43 //课程的学生名单
44 class Roster {
45 public:
46     void addStudent( Student& s );
47     void print( ) const;
48 private:
49     std::vector<std::string> mStudentNames { }; //学生名单
50 };
51 //注册类
52 class Register {
53 public:
54     void enroll( Student& s, Course& c );
55 };

```

相应的实现为:

```

1 //学生(Student)的实现
2 inline Student::Student( const std::string& name )
3     :mName(name ) {
4     mpSchedule = new Schedule;
5 }

```

```

6  inline Student::~~Student() {
7      delete mpSchedule;
8  }
9  inline void Student::print( ) const {
10     std::cout << "学生: "<<mName << "\n";
11     mpSchedule->print( );
12 }
13 //课程(Course)的实现
14 inline Course::Course( const std::string& courseName,int id )
15     :mName( courseName ),mID(id ) {
16     mpRoster = new Roster;
17 }
18 inline Course::~~Course( ) {
19     delete mpRoster;
20 }
21 inline void Course::print( ) const {
22     std::cout << "课程: "<< mName << " "<< mID<<"\n";
23     mpRoster->print( );
24 }
25 //学生选课表
26 inline void Schedule::addCourse( Course& course ) {
27     mCourseNames.emplace_back( course.name() );
28 }
29 inline void Schedule::print( ) const {
30     std::cout << "课表:\n";
31     for (auto& str : mCourseNames)
32         std::cout << str << "\n";
33     std::cout << "\n";
34 }
35 //课程的花名册
36 inline void Roster::addStudent( Student& s ) {
37     mStudentNames.emplace_back( s.name( ) );
38 }
39 inline void Roster::print( ) const {
40     std::cout << "学生名单:\n";
41     for (auto& str : mStudentNames)
42         std::cout << str << "\n";
43     std::cout << "\n";
44 }
45 inline void Register::enroll( Student& s, Course& c ) {
46     s.schedule( )->addCourse( c );
47     c.roster( )->addStudent( s );
48 }

```

示例性主函数及输出如下：

```
1  int main( ) {  
2      Student s1( "张三" );  
3      Student s2( "李四" );  
4      Student s3( "王五" );  
5  
6      Course  c1( "大学语文", 101 );  
7      Course  c2( "现代金融", 201 );  
8      Course  c3( "冷链物流", 205 );  
9  
10     Register reg;  
11     reg.enroll( s1, c1 );//注册 s1 选课 c1  
12     reg.enroll( s1, c2 );//注册 s1 选课 c2  
13     reg.enroll( s2, c2 );//注册 s2 选课 c2  
14     reg.enroll( s3, c3 );//注册 s3 选课 c3  
15  
16     s1.print( );  
17     s2.print( );  
18     s3.print( );  
19  
20     c1.print( );  
21     c2.print( );  
22     c3.print( );  
23 }
```

输出为：

学生：张三

课表：

大学语文

现代金融

学生：李四

课表：

现代金融

学生：王五

课表：

冷链物流

课程：大学语文

学生名单：

张三

课程：现代金融

学生名单：

张三

李四

课程：冷链物流

学生名单：

王五

14.5.9 关联及关联类

现定义男人(Man)和女人(Women)类，男人和女人之间可以存在婚姻关系，两个"未婚"的男人和女人可以结婚，"已婚"的男人或女人也可以离婚。男人和女人之间是通过婚姻关系联系起来的，如果这种联系只需要表示为"有"或"没有"，那么男人和女人之间就是一种双向的普通关联关系，采用组合、聚合或依赖关系都不适合。此时，Man 和 Woman 的定义和实现可如下：

```

1  #pragma once
2  #include <iostream>
3  #include <string>
4  class Woman;
5  //男人
6  class Man {
7      friend class Woman;
8  public:
9      Man( const std::string& name ) : mName( name ) {}
10     bool isMarried( ) const { return mpWife; }
11     void marriage( Woman& w );    //结婚
12     void divorce( );              //离婚
13     void info( ) const;           //显式信息
14 private:
15     const std::string mName;
16     Woman* mpWife { nullptr };
17 };
18 //女人
19 class Woman {
20     friend class Man;
21 public:
22     Woman( const std::string& name ) : mName( name ) {}
23     bool isMarried( ) const { return mpHusband; }
24     void marriage( Man& m );      //结婚

```

```

25     void divorce( );           //离婚
26     void info( ) const;        //显式信息
27 private:
28     const std::string mName;
29     Man* mpHusband { nullptr };
30 };
31
32 //内联实现-Man
33 inline void Man::marriage( Woman& w ) {
34     if (isMarried( ) || w.isMarried( )) {
35         std::cout << "不符合结婚条件, 不予批准\n";
36         return;
37     }
38     mpWife = &w;
39     w.mpHusband = this;
40     std::cout << mName << "结婚了,新娘是" << mpWife->mName << "\n";
41 }
42 inline void Man::divorce( ) {
43     if (isMarried( )) {
44         std::cout << mName << "与" << mpWife->mName << "离婚了\n";
45         mpWife->mpHusband = nullptr;
46         mpWife = nullptr;
47     }
48 }
49 inline void Man::info( ) const {
50     std::cout << "男人-姓名-" << mName << "\n";
51     if (!isMarried( )) {
52         std::cout << "婚姻状况:未婚\n";
53     } else {
54         std::cout << "婚姻状况:已婚\t 妻子姓名:" << mpWife->mName << "\n";
55     }
56 }
57
58 //内联实现-Woman
59 inline void Woman::marriage( Man& m ) {
60     if (isMarried( ) || m.isMarried( )) {
61         std::cout << "不符合结婚条件, 不予批准\n";
62         return;
63     }
64     mpHusband = &m;
65     m.mpWife = this;
66     std::cout << mName << "结婚了,新郎是" << mpHusband->mName << "\n";
67 }
68 inline void Woman::divorce( ) {

```



```

69     if (isMarried( )) {
70         std::cout << mName << "与" << mpHusband->mName << "离婚了\n";
71         mpHusband->mpWife = nullptr;
72         mpHusband = nullptr;
73     }
74 }
75 inline void Woman::info( ) const {
76     std::cout << "女人-姓名-" << mName << "\n";
77     if (!isMarried( )) {
78         std::cout << "婚姻状况:未婚\n";
79     } else {
80         std::cout << "婚姻状况:已婚\t 丈夫姓名:" << mpHusband->mName << "\n";
81     }
82 }

```

主函数:

```

1  int main( ) {
2      Man m1( "陆游" );
3      Woman w1( "唐琬" );
4
5      m1.marriage( w1 ); //陆游与唐琬结婚了
6      m1.info( );
7      w1.info( );
8
9      m1.divorce( ); //陆游与唐琬离婚了
10     w1.info( );
11     m1.info( );
12
13     Man m2( "赵士程" );
14     w1.marriage( m2 ); //唐琬与赵士程结婚了
15     m2.info( );
16     w1.info( );
17 }

```

输出内容:

```

陆游结婚了,新娘是唐琬
男人-姓名-陆游
婚姻状况:已婚   妻子姓名:唐琬
女人-姓名-唐琬
婚姻状况:已婚   丈夫姓名:陆游
陆游与唐琬离婚了
女人-姓名-唐琬
婚姻状况:未婚
男人-姓名-陆游

```

```
婚姻状况:未婚
唐琬结婚了,新郎是赵士程
男人-姓名-赵士程
婚姻状况:已婚    妻子姓名:唐琬
女人-姓名-唐琬
婚姻状况:已婚    丈夫姓名:赵士程
```

上面的代码仅体现了最简易的婚姻关联形式，一旦婚姻关系需要承载更多细节信息，上述设计便不再适用，例如记录婚姻起始日期、举办金婚纪念活动等业务场景。

由于婚姻关系本身往往包含专属的属性数据与业务行为，这类信息既不宜封装到男人类中，也不适合放入女人类中。此时可将婚姻关系独立抽象，作为男人与女人之间关联的载体，这就是关联类的设计思想。若把婚姻关系抽象为关联类 `Couple`，男人与女人便不再直接建立双向关联，而是各自与 `Couple` 类形成双向关联，通过关联类实现两个实体间的间接关联。同时 `Couple` 类可自行扩展专属的属性与行为，承载婚姻关系独有的数据和业务逻辑。

在使用关联类 `Couple` 时，还需要考虑其实例的创建与销毁权责问题。如果交由男人类或女人类自行管理，容易出现重复创建、重复销毁等不合理问题。更合理的设计方式是，单独设计一个专门管理婚姻关联的管理类，如 `CoupleMgr`。由于全局只需一份婚姻关系管理实例，可将 `CoupleMgr` 设计为单例类，统一负责 `Couple` 对象的生命周期管理。

采用关联类 `Couple` 的类图可如图 14-2 所示。

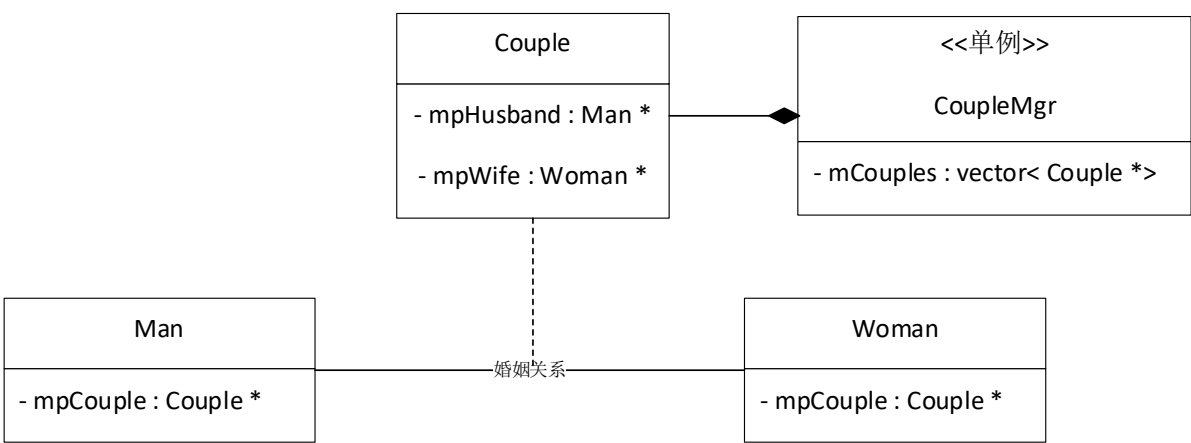


图 14-2 Couple 关联类的类图

下面是重新设计的代码：

```
1 //Man、Woman、Couple、CoupleMgr 类的定义
2 #pragma once
3 #include <iostream>
```

```

4  #include <string>
5  #include <vector>
6
7  class Man;
8  class Woman;
9  class Couple;
10 //单例的 Couple 管理器
11 class CoupleMgr {
12 public:
13     static CoupleMgr& getInstance( ) {
14         static CoupleMgr mgr;
15         return mgr;
16     }
17     ~CoupleMgr( ); //析构函数不能私有
18 private:
19     CoupleMgr( ) = default;
20     CoupleMgr( const CoupleMgr& ) = delete;
21     CoupleMgr( CoupleMgr&& ) = delete;
22     CoupleMgr& operator=( const CoupleMgr& ) = delete;
23 public:
24     Couple* createCouple( Man& m, Woman& w );
25 private:
26     std::vector<Couple*> mCouples;
27 };
28 //婚姻关系抽象为关联类 Couple
29 class Couple {
30 public:
31     Couple( Man& m, Woman& w );
32     void divorce( );
33     void show( ) const;
34 public:
35     Man* mpHusband { nullptr };
36     Woman* mpWife { nullptr };
37     int mYear { 2025 };
38     std::string mCity { "中国" };
39 };
40 //男人
41 class Man {
42     friend class Couple;
43 public:
44     Man( const std::string& name ) : mName( name ) {}
45 public:
46     bool isMarried( ) const { return bool( mpCouple ); }
47     Couple* marriage( Woman& w );

```

```

47     void divorce( );
48     void info( ) const;
49 public://getters
50     const std::string& name( ) const { return mName; };
51 private:
52     const std::string mName;
53     Couple* mpCouple { nullptr };
54 };
55 //女人
56 class Woman {
57     friend class Couple;
58 public:
59     Woman( const std::string& name ) : mName( name ) {}
60 public:
61     bool isMarried( ) const { return bool( mpCouple ); }
62     Couple* marriage( Man& m );
63     void divorce( );
64     void info( ) const;
65 public://getters
66     const std::string& name( ) const { return mName; };
67 private:
68     const std::string mName;
69     Couple* mpCouple { nullptr };
70 };

```

对应的实现为:

```

1  //内联实现-CoupleMgr
2  inline CoupleMgr::~CoupleMgr( ) {
3      for (auto item : mCouples) delete item;
4  }
5  inline Couple* CoupleMgr::createCouple( Man& m, Woman& w ) {
6      Couple* couple = new Couple( m, w );
7      mCouples.emplace_back( couple );
8      return couple;
9  }
10 //内联实现-Couple
11 inline Couple::Couple( Man& m, Woman& w )
12     :mpHusband { &m }, mpWife { &w } {
13     mpHusband->mpCouple = this; //设置丈夫中的 mpCouple
14     mpWife->mpCouple = this; //设置妻子中的 mpCouple
15 }
16 inline void Couple::divorce( ) {
17     mpHusband->mpCouple = nullptr;
18     mpWife->mpCouple = nullptr;

```

```

19     mpHusband = nullptr;
20     mpWife = nullptr;
21 }
22 inline void Couple::show( ) const {
23     std::cout << "公元" << mYear << "年,";
24     std::cout << mpHusband->name( ) << "(丈夫)与";
25     std::cout << mpWife->name( ) << "(妻子)在";
26     std::cout << mCity << "结婚\n";
27 }
28 //内联实现-Man
29 inline Couple* Man::marriage( Woman& w ) {
30     if (isMarried( ) || w.isMarried( )) {
31         std::cout << "不符合结婚条件, 不予批准\n";
32         return nullptr;
33     }
34     mpCouple = CoupleMgr::getInstance( ).createCouple( *this, w );
35     std::cout << mName << "结婚了,";
36     std::cout << "新娘是" << mpCouple->mpWife->name( ) << "\n";
37     return mpCouple;
38 }
39 inline void Man::divorce( ) {
40     if (isMarried( )) {
41         std::cout << mName << "与" << mpCouple->mpWife->name( ) << "离婚了\n";
42         mpCouple->divorce( );
43     }
44 }
45 inline void Man::info( ) const {
46     std::cout << "男人-姓名-" << mName << "\n";
47     if (!isMarried( )) {
48         std::cout << "婚姻状况:未婚\n";
49     } else {
50         std::cout << "婚姻状况:已婚\t";
51         Std::cout << "妻子姓名:" << mpCouple->mpWife->name( ) << "\n";
52     }
53 }
54 //内联实现-Woman
55 inline Couple* Woman::marriage( Man& m ) {
56     if (isMarried( ) || m.isMarried( )) {
57         std::cout << "不符合结婚条件, 不予批准\n";
58         return nullptr;
59     }
60     mpCouple = CoupleMgr::getInstance( ).createCouple( m, *this );
61     std::cout << mName << "结婚了,";
62     std::cout << "新郎是" << mpCouple->mpHusband->name( ) << "\n";

```

```

63     return mpCouple;
64 }
65 inline void Woman::divorce( ) {
66     if (isMarried( )) {
67         std::cout << mName << "与" << mpCouple->mpHusband->name( ) << "离婚了\n";
68         mpCouple->divorce( );
69     }
70 }
71 inline void Woman::info( ) const {
72     std::cout << "女人-姓名-" << mName << "\n";
73     if (!isMarried( )) {
74         std::cout << "婚姻状况:未婚\n";
75     } else {
76         std::cout << "婚姻状况:已婚\t";
77         std::cout << "丈夫姓名:" << mpCouple->mpHusband->name( ) << "\n";
78     }
79 }

```

主函数:

```

1  int main( ) {
2      Man m1( "陆游" );
3      Woman w1( "唐琬" );
4
5      auto c1 = m1.marriage( w1 ); //陆游与唐琬结婚了
6      c1->mYear = 1144;
7      c1->mCity = "绍兴";
8      c1->show( );
9      m1.info( );
10     w1.info( );
11
12     m1.divorce( );                //陆游与唐琬离婚了
13     m1.info( );
14     w1.info( );
15
16     Man m2( "赵士程" );
17     auto c2 = w1.marriage( m2 ); //唐琬与赵士程结婚了
18     c2->mYear = 1147;
19     c2->mCity = "绍兴";
20     c2->show( );
21     w1.info( );
22     m2.info( );
23 }

```

输出为:

陆游结婚了,新娘是唐琬
 公元 1144 年,陆游(丈夫)与唐琬(妻子)在绍兴结婚
 男人-姓名-陆游
 婚姻状况:已婚 妻子姓名:唐琬
 女人-姓名-唐琬
 婚姻状况:已婚 丈夫姓名:陆游
 陆游与唐琬离婚了
 男人-姓名-陆游
 婚姻状况:未婚
 女人-姓名-唐琬
 婚姻状况:未婚
 唐琬结婚了,新郎是赵士程
 公元 1147 年,赵士程(丈夫)与唐琬(妻子)在绍兴结婚
 女人-姓名-唐琬
 婚姻状况:已婚 丈夫姓名:赵士程
 男人-姓名-赵士程
 婚姻状况:已婚 妻子姓名:唐琬

14.5.10 包覆类

现定义二叉树中的结点 Node 如下:

```
1 struct Node {
2     ~Node( ) { delete left; delete right; }
3     int num;
4     Node * left
5     Node * right;
6 };
```

但在每次访问结点时,必须保证上级结点不为空,例如:

```
1 int main( ) {
2     Node* p1 = new Node { 1 };
3     Node* p2 = new Node { 2 };
4     Node* pRoot = new Node { 3, p1 , p2 };
5
6     //下条语句运行正确
7     int num1 = pRoot->left->num;
8     //下条语句运行异常。因为 pRoot->left->right 为 nullptr, 所以取 num 时出现异常
9     int num2 = pRoot->left->right->num;
10    delete pRoot;
```

```
11 }
```

从代码可见，遍历 Node 的不同路径时，每一次结点访问都需要先判断指针，代码中会出现大量的条件判断，既不优雅也容易出错。为简化结点访问，避免每次存取 left、right 或者 num 时都检查指针是否为空，可引入如下的 Wrapper 类：

```
1  class Wrapper {
2  public:
3      Wrapper( Node* pNode ) :mp( pNode ) { }
4      Wrapper( const Wrapper& rhs ) = delete;
5      Wrapper( Wrapper&& rhs ) noexcept : mp( nullptr ) { std::swap( mp, rhs.mp ); }
6      Wrapper& operator=( const Wrapper& ) = delete;
7      operator bool( ) const { return !mp; }
8  public:
9      //读取
10     Node* get( )          { return mp; }
11     const Node* get( ) const { return mp; }
12     Wrapper left( )       { return Wrapper( mp ? mp->left : nullptr ); }
13     const Wrapper left( ) const { return Wrapper( mp ? mp->left : nullptr ); }
14     Wrapper right( )      { return Wrapper( mp ? mp->right : nullptr ); }
15     const Wrapper right( ) const { return Wrapper( mp ? mp->right : nullptr ); }
16     const int* numPtr( ) const { return mp ? &mp->num : nullptr; }
17     int* numPtr( )        { return mp ? &mp->num : nullptr; }
18
19     //写入
20     Wrapper& setNum( int n ) {
21         if (mp) mp->num = n;
22         return *this;
23     }
24     Wrapper& setLeft( Node* pLeft ) {
25         if ( mp ) {
26             delete mp->left;
27             mp->left = pLeft;
28         }
29         return *this;
30     }
31     Wrapper& setRight( Node* pRight ) {
32         if (mp) {
33             delete mp->right;
34             mp->right = pRight;
35         }
36         return *this;
37     }
```



```

38 private:
39     Node* mp;
40 };

```

使用覆盖类 Wrapper 就可以简化 Node 结点的访问了，例如：

```

1  #include <iostream>
2  using namespace std;
3  int main( ) {
4      Node* p1 = new Node { 1 };
5      Node* p2 = new Node { 2 };
6      Node* pRoot = new Node { 3,p1,p2 };
7      Node* p4 = new Node { 4 };
8      Node* p5 = new Node { 5 };
9
10     {
11         cout << "Wrapper 对象:\n";
12         Wrapper wrapper( pRoot );
13         auto ptr1 = wrapper.right( ).numPtr( );
14         if ( ptr1 ) cout << *ptr1 << endl;    //条件满足,输出 2
15
16         auto ptr2 = wrapper.left( ).setLeft( p4 ).setRight(p5).left( ).numPtr( );
17         if (ptr2) cout << *ptr2 << endl;  //条件满足,输出 4
18
19         auto ptr3 = wrapper.left( ).right( ).right( ).numPtr( );
20         if (ptr3) cout << *ptr3 << endl;  //条件不满足,无输出
21     }
22     {
23         cout << "const Wrapper 对象:\n";
24         const Wrapper constWrapper( pRoot );
25         auto ptr1 = constWrapper.right( ).numPtr( );
26         if (ptr1) cout << *ptr1 << endl; //条件满足,输出 2
27
28         auto ptr2 = constWrapper.left( ).right( ).right( ).numPtr( );
29         if (ptr2) cout << *ptr2 << endl; //条件不满足,无输出
30
31         //下条语句语法报错: setNum 要求对象是非 const 对象
32         //auto ptr3 = constWrapper.left( ).setNum( 7 ).numPtr( );
33         //if (ptr3) cout << *ptr3 << endl;
34     }
35
36     delete pRoot;
37 }

```

在上述示例中，直接使用了 Wrapper(root).left().right().right().numPtr()的形式访问结点，而

且在这种链式访问过程中不再需要频繁判断指针是否为空，简化了对树中各节点的访问。

例子中 `Wrapper::numPtr()` 函数返回整数指针，通过返回值是否为空，区分是否是合法值。在 C++17 中，也可以使用 `std::optional` 作为返回值，有关 `std::optional` 可详见异常处理一章。此外，在 `Wrapper` 类中，这里禁止了普通的拷贝构造和赋值函数，但又自定义了移动拷贝，请思考目的是什么？如果这几个函数都禁用，是否可以？如果都不禁用且都使用缺省的，是否可以？进一步请参考本例，请定义并实现一个适用于 `const Node *` 型指针的 `CWrapper` 类。

14.5.11 Json 格式

JSON(JavaScript Object Notation)也称 JavaScript 对象表示法，是一种轻量级的文本数据交换格式，由于便于阅读和编写，成为广泛使用的数据格式。JSON 有两种数据结构，一种是对象，它保存一个无序的名称/值对集合，一个对象以左括号 `{` 开始，右括号 `}` 结束。每个键后跟一个冒号 `:`，名称/值对使用逗号 `,` 分隔。另一种数据结构是数组，它用于保存值的有序集合。一个数组以左中括号 `[` 开始，右中括号 `]` 结束，值之间使用逗号 `,` 分隔。JSON 中的值可以是字符串、数值、布尔值、`null`、对象或者数组，它们是可以嵌套。

对于 JSON 数据格式，下面针对其主要功能，讨论并给出相关类的设计和实现。

JSON 中的值可以是多种，为此 `Value` 类中可以使用 `union` 或使用 `void*` 指针存放值。例如：

```

1  enum class VType { NULL_T=0,OBJECT,ARRAY,INTEGER,BOOLEAN,DOUBLE,STRING };
2  class Value {
3      VType mType;
4      union {
5          Object * pObj;
6          Array * pArr;
7          String * pStr;
8          int      num;
9          bool      truefalse;
10         double   val;
11     };
12 };
13 或者
14 class Value {
15     VType mType;
16     void * mpVal;
17 };

```

这里，我们用第二种形式定义 `Value` 类，如下：

```

1  //Json.h
2  #pragma once
3  #include <string>
4  #include <vector>
5  #include <initializer_list>
6  #include <type_traits>
7
8  namespace JsonNS {
9      //值类型枚举
10     enum class VType { NULL_T = 0, OBJECT, ARRAY,
                        STRING, INTEGER, BOOLEAN, DOUBLE };
11     //由于 map 不能按加入顺序输出,这里采用 vector 存储 Object
12     class Value;
13     using Object = std::vector<std::pair<std::string, Value> >;
14     using Array = std::vector<Value>;
15
16     class Value {
17     public://构造
18         Value( const char* s = nullptr );
19         Value( const std::string& s );
20         Value( std::initializer_list<Value> list );
21         Value( int n );
22         Value( double v );
23         Value( bool b );
24     public: //拷贝
25         Value( const Value& v );
26         Value( Value&& v ) noexcept;
27     public: //赋值
28         Value& operator=( const Value& v );
29         Value& operator=( Value&& v ) noexcept;
30     public://析构
31         ~Value( );
32     public://数组或对象分量
33         Value& operator[]( size_t index );
34         Value& operator[]( const std::string& key );
35     public: //输出、序列化、反序列化
36         std::string dump( ) const; //格式序列化
37         //std::string stringify( ) const; //序列化, 未实现
38         //static std::string serialize( const Value& v ); //序列化, 未实现
39         //static Value& parse( const std::string& str ); //反序列化, 未实现
40     private://辅助函数
41         void reset( ); //重置为 NULL_T 类型
42         void beautifier( std::stringstream& ss, size_t indents = 0 ) const;
43     private:

```

```

44     VType mType; //值类型
45     void* mpVal; //指向值的指针
46 };
47 }

```

这里的 Object 存放多个键值对，而 Array 直接存放多个值。Vaule 类通过数据成员 mpVal 指向由 mType 表明的值，由于 mpVal 的类型是 void*，可以表示指向任意类型的指针，当需要取得指向的值时，再通过 static_cast 转换运算符取得。因此，Object、Array 于 Value 之间的关系都是双向的，Object 组合 Value，Array 组合 Value，Value 关联 Object 和 Array 类，对应的 UML 类图如下：

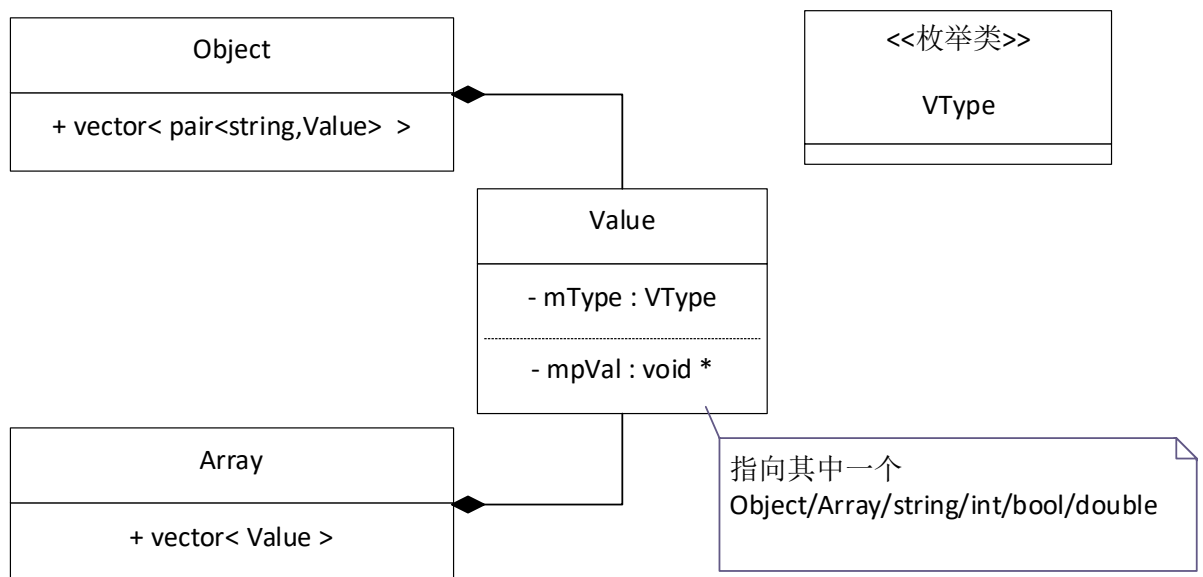


图 14-3 Json 格式定义中各类之间的关系

下面给出具体实现：

```

1  //Json.cpp
2  #include <string>
3  #include <sstream> //for stringstream
4  #include "Json.h"
5  //构造
6  JsonNS::Value::Value( const char* s ) : mType( VType::NULL_T ), mpVal( nullptr ) {
7      if (s) { //s 非空才为字符串
8          mType = VType::STRING;
9          mpVal = new std::string( s );
10     }
11 }
12 JsonNS::Value::Value( const std::string& s )
13     : mType( VType::STRING ), mpVal( new std::string( s ) ) { }
14 JsonNS::Value::Value( std::initializer_list<Value> list )
15     : mType( VType::ARRAY ), mpVal( new Array ) {

```

```

16     Array& arr = *static_cast<Array*>( mpVal );
17     arr.assign( list );
18 }
19 JsonNS::Value::Value( int n )
20     : mType( VType::INTEGER ), mpVal( new int( n ) ) { }
21 JsonNS::Value::Value( double v )
22     : mType( VType::DOUBLE ), mpVal( new double( v ) ) { }
23 JsonNS::Value::Value( bool b )
24     : mType( VType::BOOLEAN ), mpVal( new bool( b ) ) { }
25 //拷贝
26 JsonNS::Value::Value( const Value& v ) : mType( v.mType ), mpVal( nullptr ) {
27     switch (mType) {
28     case VType::INTEGER:
29         mpVal = new int( *static_cast<int*>( v.mpVal ) );
30         break;
31     case VType::DOUBLE:
32         mpVal = new double( *static_cast<double*>( v.mpVal ) );
33         break;
34     case VType::BOOLEAN:
35         mpVal = new bool( *static_cast<bool*>( v.mpVal ) );
36         break;
37     case VType::STRING:
38         mpVal = new std::string( *static_cast<std::string*>( v.mpVal ) );
39         break;
40     case VType::ARRAY:
41         mpVal = new Array( *static_cast<Array*>( v.mpVal ) );
42         break;
43     case VType::OBJECT:
44         mpVal = new Object( *static_cast<Object*>( v.mpVal ) );
45         break;
46     default:
47         break;
48     }
49 }
50 JsonNS::Value::Value( Value&& v ) noexcept
51     :mType( VType::NULL_T ), mpVal( nullptr ) {
52     std::swap( mType, v.mType );
53     std::swap( mpVal, v.mpVal );
54 }
55 //赋值
56 JsonNS::Value& JsonNS::Value::operator=( const Value& v ) {
57     Value temp( v );
58     std::swap( mType, temp.mType );
59     std::swap( mpVal, temp.mpVal );

```

```

60     return *this;
61 }
62 JsonNS::Value& JsonNS::Value::operator=( Value&& v ) noexcept {
63     std::swap( mType, v.mType );
64     std::swap( mpVal, v.mpVal );
65     return *this;
66 }
67 //数组分量
68 JsonNS::Value& JsonNS::Value::operator[]( size_t index ) {
69     if (mType == VType::NULL_T) {
70         mType = VType::ARRAY;
71         mpVal = new Array {};
72     }
73     if (mType != VType::ARRAY) throw std::string( "Type is not ARRAY!" );
74
75     Array& arr = *static_cast<Array*>( mpVal );
76     //数组中无此索引, 添加空元素
77     for (size_t pos = arr.size( ); pos <= index; pos++)
78         arr.emplace_back( Value { } );
79     return arr[ index ];
80 }
81 //对象的键值对分量
82 JsonNS::Value& JsonNS::Value::operator[]( const std::string& key ) {
83     if (mType == VType::NULL_T) {
84         mType = VType::OBJECT;
85         mpVal = new Object {};
86     }
87     if (mType != VType::OBJECT) throw std::string( "Type is not OBJECT!" );
88
89     auto& obj = *static_cast<Object*>( mpVal );
90     auto it = std::find_if( obj.begin( ), obj.end( ),
91                             [&key]( std::pair<std::string, Value> pair ) {
92                                 return ( pair.first == key );
93                             } );
94     if (it == obj.end( )) {
95         return obj.emplace_back( std::pair( key, JsonNS::Value { } ) ).second;
96     }
97     return it->second;
98 }
99 //析构
100 JsonNS::Value::~~Value( ) {
101     reset( );
102 }
103 //输出

```

```

104     std::string JsonNS::Value::dump( ) const {
105         std::stringstream ss;
106         beautifier( ss, 0 );
107         return ss.str( );
108     }
109     //辅助
110     void JsonNS::Value::reset( ) {
111         switch (mType) {
112             case VType::NULL_T:
113                 break;
114             case VType::INTEGER:
115                 delete static_cast<int*>( mpVal );
116                 break;
117             case VType::DOUBLE:
118                 delete static_cast<double*>( mpVal );
119                 break;
120             case VType::BOOLEAN:
121                 delete static_cast<bool*>( mpVal );
122                 break;
123             case VType::STRING:
124                 delete static_cast<std::string*>( mpVal );
125                 break;
126             case VType::ARRAY:
127                 delete static_cast<Array*>( mpVal );
128                 break;
129             case VType::OBJECT:
130                 delete static_cast<Object*>( mpVal );
131                 break;
132             default:
133                 break;
134         }
135         mType = VType::NULL_T;
136         mpVal = nullptr;
137     }
138     void JsonNS::Value::beautifier( std::stringstream& ss, size_t indents ) const {
139         switch (mType) {
140             case VType::NULL_T:
141                 ss << "null";
142                 break;
143             case VType::INTEGER:
144                 ss << *static_cast<int*>( mpVal );
145                 break;
146             case VType::DOUBLE:
147                 ss << *static_cast<double*>( mpVal );

```

```

148         break;
149     case VType::BOOLEAN:
150         ss << ( *static_cast<bool*>( mpVal ) ? "true" : "false" );
151         break;
152     case VType::STRING:
153         ss << "\"" << *static_cast<std::string*>( mpVal ) << "\"";
154         break;
155     case VType::ARRAY:
156     {
157         ss << "[\n";
158         Array& arr = *static_cast<Array*>( mpVal );
159         size_t total = arr.size( );
160         for (size_t pos = 0; pos < total; pos++) {
161             ss << std::string( indents + 4, ' ' );
162             arr[ pos ].beautifier( ss, indents + 4 );
163             if (pos + 1 < total) ss << ",";
164             ss << "\n";
165         }
166         ss << std::string( indents, ' ' ) << "];";
167     }
168     break;
169     case VType::OBJECT:
170     {
171         ss << "{\n";
172         Object& obj = *static_cast<Object*>( mpVal );
173         size_t total = obj.size( );
174         size_t pos = 0;
175         for (auto& field : obj) {
176             ss << std::string( indents + 4, ' ' );
177             ss << "\"" << field.first << "\" : ";
178             size_t offset = 4; // +1 + field.first.size( ) + 4;
179             field.second.beautifier( ss, indents + offset );
180             if (++pos < total) ss << ",";
181             ss << "\n";
182         }
183         ss << std::string( indents, ' ' ) << "}";
184     }
185     break;
186     default:
187         break;
188 }
189 }

```

使用 `JsonNS::Value` 类，就可以创建 `Json` 格式的数据了。例如：


```

1  #include <iostream>
2  #include "Json.h"
3  int main( ) {
4      using namespace std;
5      using Json = JsonNS::Value;
6
7      Json j1;
8      j1[ "姓名" ] = "张三";
9      j1[ "年龄" ] = 22;
10     j1[ "帅哥/美女" ] = true;
11     j1[ "第一专业" ] = "计算机科学与技术";
12     j1[ "第二专业" ] = nullptr;
13     j1[ "特长" ][ "体育" ] = { "游泳","足球" };
14     j1[ "特长" ][ "专业" ] = { "信息技术","动漫制作" };
15
16     Json j2;
17     j2[ "高等数学" ] = 92.5;
18     j2[ "面向对象程序设计" ] = 97.5;
19     j2[ "其它课程" ] = { 90,91,88.7 };
20     j1[ "成绩" ] = j2;
21
22     cout << "j1 = " << j1.dump( ) << endl;
23 }

```

输出结果为:

```

j1 = {
    "姓名" : "张三",
    "年龄" : 22,
    "帅哥/美女" : true,
    "第一专业" : "计算机科学与技术",
    "第二专业" : null,
    "特长" : {
        "体育" : [
            "游泳",
            "足球"
        ],
        "专业" : [
            "信息技术",
            "动漫制作"
        ]
    },
    "成绩" : {
        "高等数学" : 92.5,

```

```
"面向对象程序设计" : 97.5,  
"其它课程" : [  
    90,  
    91,  
    88.7  
]  
}  
}
```